SIDDAGANGA INSTITUTE OF TECHNOLOGY, TUMAKURU-572103

(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)

Project Report on

"Memory Augmented Meta Hypernetwork For Adaptive Anomaly Detection On Edge Devices"

submitted in partial fulfillment of the requirement for the completion of V semester of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE & ENGINEERING

Submitted by

A S Sameeksha (1SI23AD001)

Bhuvana G P (1SI23AD008)

under the guidance of

Dr. Bhargavi K

Associate professor

Department of CSE

SIT, Tumakuru-03

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2025-26

SIDDAGANGA INSTITUTE OF TECHNOLOGY, TUMAKURU-572103

(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the mini project work entitled "MEMORY AUGMENTED META HYPER NETWORK FOR ADAPTIVE ANOMALY DETECTION ON EDGE COMPUTING" is a bonafide work carried out by A S Sameeksha (1SI23AD001), Bhuvana G P (1SI23AD008) in partial fulfillment for the completion of V Semester of Bachelor of Engineering in Computer Science and Engineering from Siddaganga Institute of Technology, an autonomous institute under Visvesvaraya Technological University, Belagavi during the academic year 2025-26. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The Mini project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the Bachelor of Engineering degree.

Dr. Bhargavi K
Associate Professor
Dept. of CSE
SIT,Tumakuru-03

Head of the Department
Dept. of CSE
SIT,Tumakuru-03

External viva:

Names of the Examiners **Signature with date**

1.

2.

ACKNOWLEDGEMENT

We offer our humble pranams at the lotus feet of **His** Holiness, **Dr. Sree Sree Sivakumara Swamigalu**, Founder President and **His** Holiness, **Sree Sree Siddalinga Swamigalu**, President, Sree Siddaganga Education Society, Sree Siddaganga Math for bestowing upon their blessings.

We deem it as a privilege to thank **Dr. Shivakumaraiah**, CEO, SIT, Tumakuru, and **Dr. S V Dinesh**, Principal, SIT, Tumakuru for fostering an excellent academic environment in this institution, which made this endeavor fruitful.

We would like to express our sincere gratitude to **Dr. N R Sunitha**, Professor and Head, Department of CSE, SIT, Tumakuru for his encouragement and valuable suggestions.

We thank our guide **Dr. Bhargavi K**, Associate professor, Department of Computer Science Engineering, SIT, Tumakuru for the valuable guidance, advice and encouragement.

Bhuvana G P (1SI23AD008)
A S Sameeksha (1SI23AD001)

Course Outcomes

CO1: To identify a problem through literature survey and knowledge of contemporary engineering technology.

CO2: To consolidate the literature search to identify issues/gaps and formulate the engineering problem

CO3: To prepare project schedule for the identified design methodology and engage in budget analysis, and share responsibility for every member in the team

CO4: To provide sustainable engineering solution considering health, safety, legal, cultural issues and also demonstrate concern for environment

CO5: To identify and apply the mathematical concepts, science concepts, engineering and management concepts necessary to implement the identified engineering problem

CO6: To select the engineering tools/components required to implement the proposed solution for the identified engineering problem

CO7: To analyze, design, and implement optimal design solution, interpret results of experiments and draw valid conclusion

CO8: To demonstrate effective written communication through the project report, the one-page poster presentation, and preparation of the video about the project and the four page IEEE/Springer/ paper format of the work

CO9: To engage in effective oral communication through power point presentation and demonstration of the project work

CO10:To demonstrate compliance to the prescribed standards/ safety norms and abide by the norms of professional ethics

CO11: To perform in the team, contribute to the team and mentor/lead the team

Attainment level:- 1: Slight (low) 2: Moderate (medium) 3: Substantial (high)

POs: PO1: Engineering Knowledge, PO2: Problem analysis, PO3: Design/Development of solutions, PO4: Conduct investigations of complex problems, PO5: Engineering tool usage, PO6: Engineer and the world, PO7: Ethics, PO8: Individual and collaborative team work, PO9: Communication, PO10: Project management and finance, PO11: Lifelong learning

PSO1:Computer based systems development,PSO2:Software development,PSO3:Computer Communications and Internet applications

CO-PO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO-1											3		3	
CO-2		3		3									3	
CO-3										3	3		3	
CO-4						3	3						3	
CO-5	3	3											3	
CO-6					3								3	
CO-7		3	3	3									3	
CO-8									3				3	
CO-9									3				3	
CO-10							3						3	
CO-11								3					3	

Abstract

The rapid growth of the Internet of Things (IoT) and edge computing has resulted in the deployment of large numbers of interconnected devices that continuously generate data in dynamic and often unpredictable environments. These devices are exposed to challenges such as sensor noise, environmental variations, hardware degradation, concept drift, and cyber-attacks, making reliable anomaly detection essential for maintaining system security and operational stability. Traditional machine learning and deep learning approaches often rely on static model parameters, require large quantities of labelled data, and demand substantial computational resources, limiting their effectiveness in resource-constrained edge environments.

This project proposes a Memory-Augmented Meta-Hypernetwork (MAMHN) framework for adaptive anomaly detection in Industrial Internet of Things (IIoT) systems. The proposed architecture integrates three key components: an LSTM-based Memory Module for capturing temporal behavioural patterns, a Meta-Hypernetwork for generating context-aware dynamic parameters, and an Adaptive Main Network for anomaly classification. By leveraging memory-guided parameter generation, the framework dynamically adjusts its behaviour according to changing operating conditions while maintaining a lightweight architecture suitable for edge deployment.

The proposed framework was implemented using Python and PyTorch and evaluated on industrial time-series datasets including the SWaT dataset and the Pump Sensor Dataset. Experimental results demonstrate that the model achieves strong anomaly detection performance, obtaining an accuracy of 90.59%, precision of 90.21%, recall of 98.29%, and an F1-score of 94.08%. The framework also demonstrates the ability to adapt to changing data distributions while maintaining robust detection performance.

The results indicate that the proposed MAMHN framework provides an effective, scalable, and computationally efficient solution for real-time anomaly detection in edge-enabled Industrial IoT environments.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The rapid growth of the Internet of Things (IoT) has led to the deployment of billions of interconnected devices capable of continuously sensing, processing, and transmitting data. These devices are increasingly being integrated into critical applications such as industrial automation, smart manufacturing, healthcare monitoring, intelligent transportation, and smart city infrastructure. As the scale of IoT deployments continues to expand, enormous volumes of real-time data are generated at the network edge, creating significant challenges for timely processing, analysis, and decision-making.

To address these challenges, Edge Computing has emerged as a promising paradigm that brings computation closer to data sources. By performing data processing near IoT devices rather than relying solely on centralized cloud servers, edge computing reduces communication latency, lowers bandwidth consumption, improves privacy, and enables real-time responses. However, edge devices are typically constrained by limited computational resources, memory capacity, storage, and energy availability. These constraints make the deployment of computationally intensive machine learning models particularly challenging.

One of the most critical tasks in IoT and Industrial IoT (IIoT) environments is anomaly detection. Anomalies may arise due to equipment malfunction, sensor failures, communication disruptions, environmental changes, or cyber-attacks. Early identification of abnormal behaviour is essential for ensuring system reliability, operational safety, and cybersecurity. However, anomaly detection in edge environments remains challenging because data distributions often change over time due to concept drift, evolving operating conditions, sensor degradation, and dynamic user behaviour.

Traditional machine learning and deep learning approaches have demonstrated strong performance in anomaly detection tasks under controlled conditions. Nevertheless, most existing models rely on static parameters and require extensive retraining when deployed in changing environments. Such retraining processes consume significant computational resources and are often impractical for resource-constrained edge devices. Furthermore,

these approaches may struggle to generalize effectively when encountering previously unseen operational conditions or attack patterns.

Recent advances in adaptive learning have highlighted the potential of memory-based architectures, meta-learning, and hypernetworks for addressing these limitations. Memory-based models can capture long-term temporal dependencies present in sensor data, while meta-learning techniques enable rapid adaptation to new tasks with limited training samples. Hypernetworks further enhance adaptability by dynamically generating model parameters based on the current operating context, reducing the need for costly retraining procedures.

Motivated by these developments, this project proposes a Memory-Augmented Meta-Hypernetwork (MAMHN) framework for adaptive anomaly detection in Industrial IoT environments. The proposed architecture integrates an LSTM-based memory module, a Meta-Hypernetwork responsible for dynamic parameter generation, and an adaptive classification network for anomaly prediction. By combining temporal memory learning with context-aware parameter adaptation, the framework aims to provide accurate, efficient, and scalable anomaly detection suitable for deployment in edge computing environments.

Experimental evaluation conducted on industrial cyber-physical system datasets demonstrates that the proposed framework achieves high anomaly detection performance while supporting rapid adaptation through few-shot learning. The results indicate that MAMHN provides a practical solution for real-time anomaly detection in dynamic and resource-constrained IoT ecosystems.

1.1 Motivation

The increasing adoption of IoT and Industrial IoT technologies has resulted in the continuous generation of large-scale sensor data across diverse application domains. Smart sensors, industrial controllers, wearable devices, and environmental monitoring systems produce real-time information that must be analysed promptly to ensure safe and reliable operation. Detecting abnormal system behaviour at an early stage is crucial for preventing equipment failures, operational disruptions, and cybersecurity incidents.

Despite the success of cloud-based analytics platforms, transmitting large volumes of sensor data to centralized servers introduces latency, communication overhead, privacy concerns, and dependence on network connectivity. These limitations make cloud-centric

anomaly detection unsuitable for many time-sensitive industrial applications. Edge computing addresses these concerns by enabling local processing near data sources; however, the resource limitations of edge devices restrict the use of large and computationally expensive machine learning models.

Another significant challenge is the dynamic nature of real-world environments. Sensor readings and system behaviour often evolve over time due to environmental variations, hardware aging, maintenance activities, and changing operational conditions. Static anomaly detection models trained on historical data frequently experience performance degradation when exposed to such changes. Consequently, there is a growing need for adaptive learning systems capable of updating their behaviour without extensive retraining.

This project is motivated by the need to develop an intelligent anomaly detection framework that combines adaptability, computational efficiency, and real-time responsiveness. By integrating memory-based learning, meta-learning principles, and hypernetwork-driven parameter generation, the proposed MAMHN framework seeks to address the limitations of conventional anomaly detection approaches and enable reliable operation in dynamic edge computing environments.

1.2 Objective of the Project

The primary objective of this project is to design and implement a Memory-Augmented Meta-Hypernetwork (MAMHN) capable of performing adaptive, efficient, and real-time anomaly detection on resource-constrained IoT edge devices.

Specific Objectives

1. **Develop a lightweight anomaly detection framework** suitable for deployment on low-power and resource-constrained IoT edge devices.

2. **Implement a memory-augmented learning mechanism** capable of capturing temporal behavioural patterns from sequential sensor data.

3. **Design a Meta-Hypernetwork module** that dynamically generates context-aware model parameters to improve adaptability under changing operating conditions.

4. **Enable rapid adaptation through meta-learning principles** so that the system can adjust to new environments using limited training data.

5. **Achieve accurate real-time anomaly detection** while maintaining low computational overhead and robustness to concept drift and environmental variations.

1.3 Contributions of the Project

The major contributions of this project are summarized as follows:

1. Development of a Memory-Augmented Meta-Hypernetwork (MAMHN) architecture for adaptive anomaly detection in Industrial IoT environments.

2. Integration of an LSTM-based memory module with a hypernetwork capable of generating dynamic, context-dependent model parameters.

3. Design of an adaptive classification framework that utilizes dynamically generated parameters for anomaly prediction.

4. Implementation of a few-shot adaptation strategy that enables rapid learning in new operational environments using limited training data.

5. Experimental evaluation on industrial cyber-physical system datasets demonstrating strong anomaly detection performance and adaptability.

1.4 Organisation of the Report

The remainder of this report is organized as follows:

Chapter 1 – Introduction

This chapter presents the background, motivation, objectives, and contributions of the project. It also discusses the challenges associated with anomaly detection in IoT and edge computing environments.

Chapter 2 – Literature Survey

This chapter reviews existing research related to anomaly detection, meta-learning, hypernetworks, memory-augmented neural networks, and edge intelligence. The strengths and limitations of existing approaches are analysed to identify research gaps.

Chapter 3 – System Overview

This chapter provides a high-level overview of the proposed MAMHN framework and introduces its major components, including the memory module, Meta-Hypernetwork, and adaptive classification network.

Chapter 4 – System Design and Architecture

This chapter describes the architecture of the proposed system, including data flow, model interactions, and the complete anomaly detection pipeline.

Chapter 5 – Implementation Details

This chapter explains the implementation of the proposed framework, including data preprocessing, model design, training procedures, parameter generation mechanisms, and adaptation strategies.

Chapter 6 – Results and Analysis

This chapter presents experimental results, performance evaluation metrics, comparative analysis, and discussions on anomaly detection effectiveness and adaptation capability.

Chapter 7 – Conclusion and Future Work

This chapter summarizes the findings of the project, highlights key contributions, discusses limitations, and outlines potential directions for future research and system enhancement.

Chapter 2

Literature Survey

Jena et al. [1] propose a unified anomaly detection framework that employs knowledge distillation and quantization to create highly efficient INT-8 models suitable for deployment on memory-restricted edge devices. Although the framework achieves strong performance with reduced computation, it is unable to adapt to unseen anomaly types since the student network is limited to patterns learned during distillation. **The proposed MAMHN overcomes this limitation by enabling continual adaptation through meta-learning and memory augmentation, allowing it to detect novel behaviours not present during initial training.**

Alwaisi et al. [2] explore the deployment of lightweight intrusion detection models using TinyML techniques on IoT microcontrollers. Their study shows that decision trees offer excellent inference efficiency with minimal overhead. However, these classical models require full retraining whenever new attack patterns emerge, restricting their ability to operate effectively in dynamic or adversarial environments. **MAMHN resolves this by generating adaptive model parameters at runtime, eliminating the need for manual retraining.**

Antonini et al. [3] design an unsupervised anomaly detection system for ESP32 microcontrollers using Isolation Forests. While the system achieves fast anomaly detection on-device, limited microcontroller memory restricts model capacity, making it unable to learn complex non-linear patterns. **The proposed architecture addresses this constraint by employing a compact hypernetwork that produces lightweight, device-specific parameters without increasing storage requirements.**

Singh et al. [4] introduce a transformer-based meta-learning approach for video anomaly detection across multiple surveillance scenarios. Their model generalizes well and adapts rapidly using minimal labelled data, but its computational cost makes real-time edge deployment impractical. **In contrast, MAMHN leverages lightweight autoencoders supported by hypernetwork-generated weights, enabling practical anomaly detection on constrained devices.**

Liu et al. [5] present a memory-augmented ESN framework for EEG motor imagery classification, enabling fast cross-subject adaptation. However, the reliance on large ESN reservoirs significantly increases memory usage, limiting deployment on low-power platforms. **MAMHN avoids this issue by implementing a compact external memory module, improving scalability for edge environments.**

Kang et al. [6] propose MemAE-GAN, which incorporates an external memory bank into an autoencoder to enhance anomaly detection accuracy. Although effective, its GAN-based training and memory-query operations are computationally intensive. **The proposed MAMHN retains the advantages of memory-guided reconstruction while eliminating the heavy GAN overhead, making it suitable for edge deployment.**

Santoro et al. [7] introduce Memory-Augmented Neural Networks (MANNs), enabling rapid one-shot learning via explicit memory access. However, recurrent memory operations impose considerable computational and latency costs. **MAMHN employs simplified and efficient memory access mechanisms, maintaining adaptability without the expensive recurrent architecture.**

Zhao et al. [8] demonstrate that hypernetworks can generate task-specific parameters efficiently, supporting rapid adaptation. Nevertheless, hypernetwork training can be unstable and sensitive to initialization. **MAMHN mitigates this by generating relatively small autoencoder parameter sets, which simplifies optimization and improves training stability.**

Chauhan et al. [9] provide a comprehensive survey of hypernetwork architectures, noting challenges such as initialization sensitivity and scaling instability. **MAMHN addresses these issues by adopting a lightweight hypernetwork that produces compact parameter vectors tailored for edge anomaly detection.**

Shin et al. [10] propose HypeMeFed, a federated learning framework where a central hypernetwork personalizes client models. The system achieves effective personalization but depends heavily on cloud-based training, which limits its autonomy. **MAMHN avoids this dependency by supporting full, on-device adaptation without requiring central coordination.**

Hao et al. [11] develop a federated hypernetwork for handling diverse time-series anomaly detection tasks. However, their method struggles with scalability when the number of

clients increases. **MAMHN avoids central bottlenecks by enabling adaptation directly at the edge node.**

Zhang et al. [12] propose MetaLog, a meta-learning-based framework for cross-system log anomaly detection. Although effective in low-label settings, performance declines when domain similarity is low. **MAMHN enhances generalization through memory-driven pattern retention and meta-learned initialization.**

Wang et al. [13] introduce a multi-level graph adaptation framework that improves few-shot performance but incurs high computational overhead. **MAMHN delivers comparable adaptability with significantly lower complexity.**

Gudala et al. [14] review AI-based IoT threat detection techniques and highlight the persistent challenges of false positives, high retraining cost, and limited adaptability. **These findings reinforce the importance of the proposed MAMHN, which combines meta-learning, hypernetworks, and memory mechanisms to deliver efficient, adaptive, and real-time anomaly detection on heterogeneous edge devices.**

Tuli et al. [15] propose TranAD, a transformer-based anomaly detection framework specifically designed for multivariate time-series data. The model employs an encoder-decoder architecture with self-attention mechanisms to capture complex temporal dependencies and has been extensively evaluated on industrial control system datasets such as SWaT and WADI. TranAD achieves strong anomaly detection performance and represents a modern state-of-the-art benchmark for industrial anomaly detection. However, the self-attention operations used by transformer architectures introduce significant computational and memory overhead, making deployment on resource-constrained edge devices challenging. **The proposed MAMHN addresses this limitation by replacing computationally intensive attention mechanisms with memory-guided dynamic parameter generation, enabling adaptive anomaly detection with lower resource requirements suitable for edge computing environments.**

Wu et al. [16] introduce TimesNet, a general-purpose time-series analysis framework that transforms one-dimensional temporal data into two-dimensional representations using frequency-domain analysis. By combining Fast Fourier Transform (FFT)-based period discovery with two-dimensional convolutional processing, TimesNet achieves remarkable performance across forecasting, classification, and anomaly detection tasks. Despite its

effectiveness, the continuous execution of FFT operations and two-dimensional convolutional computations increases computational complexity and memory consumption, making deployment on low-power IoT edge devices difficult. **In contrast, MAMHN operates entirely within the temporal domain using memory augmentation and meta-learning-based parameter adaptation, providing competitive anomaly detection performance while maintaining the computational efficiency required for real-time edge deployment.**

This literature survey demonstrates that existing edge anomaly detection methods often suffer from key limitations such as static behaviour, retraining overhead, high computational cost, or inability to adapt to new patterns. **The proposed MAMHN framework integrates the strengths of meta-learning, memory augmentation, and hypernetwork-driven personalization to overcome these challenges, positioning it as a robust and scalable solution for real-world edge anomaly detection.**

Chapter 3

System Overview

The proposed Memory-Augmented Meta-Hypernetwork (MAMHN) is an adaptive anomaly detection framework developed for Industrial Internet of Things (IIoT) and edge computing environments. The primary objective of the framework is to provide accurate, efficient, and adaptive anomaly detection while operating under the resource constraints typically associated with edge devices. Industrial systems generate large volumes of multivariate time-series data from sensors, actuators, controllers, and monitoring equipment. Detecting anomalous behaviour within these data streams is essential for maintaining system reliability, operational safety, and cybersecurity.

Traditional anomaly detection approaches often rely on static models that are trained offline and deployed without further adaptation. While such models may achieve good performance under stable operating conditions, their effectiveness decreases when exposed to concept drift, changing environmental conditions, evolving attack patterns, or previously unseen operational states. To address these limitations, the proposed MAMHN framework integrates temporal memory learning, dynamic parameter generation, and adaptive classification into a unified architecture capable of learning contextual behaviour and adapting to changing environments.

The MAMHN architecture consists of three major components:

1. Memory Module (LSTM-Based Temporal Memory)
2. Meta-Hypernetwork
3. Adaptive Main Network

These components work together to learn behavioural representations from sequential sensor observations and perform anomaly classification using dynamically generated parameters. The architecture is specifically designed to support deployment in Industrial IoT environments where computational resources, memory capacity, and power availability may be limited.

The overall workflow of the proposed framework is illustrated in Figure 3.1. The architecture operates in two primary phases: an offline training phase and a runtime inference phase. During offline training, historical industrial datasets are used to learn temporal behavioural patterns and optimize the parameters of the Memory Module, Meta-Hypernetwork, and Adaptive Main Network. During runtime inference, incoming sensor data is analysed in real time, allowing the framework to generate anomaly predictions while adapting to changing operating conditions.

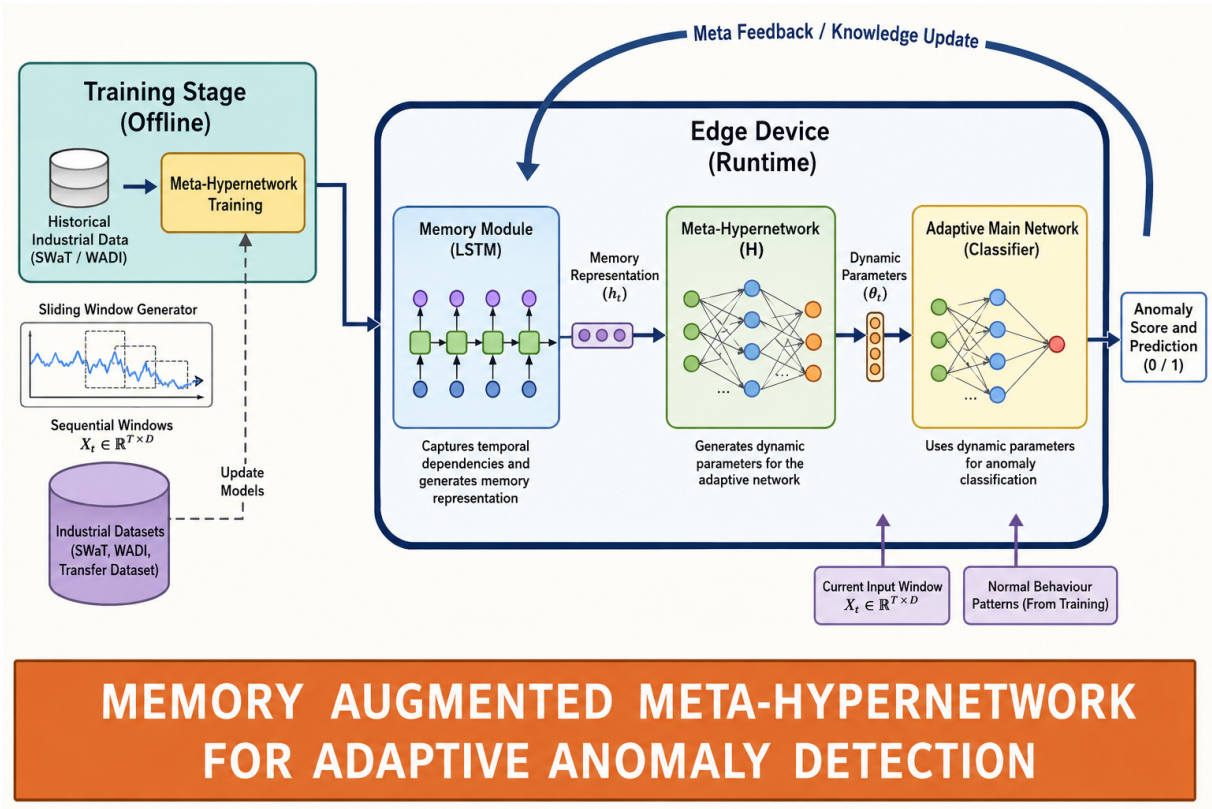

Figure 3.1: Overall System Architecture of the Proposed MAMHN Framework

3.1 Offline Training Phase

The offline training phase is responsible for learning generalized behavioural representations from historical industrial datasets. In this project, industrial control system datasets such as SWaT and WADI are utilized to train the model. These datasets contain multivariate sensor measurements collected under both normal and anomalous operating conditions.

Before training, raw sensor observations are converted into fixed-length sliding windows. Each window represents a short temporal sequence describing the behaviour of the moni-

tored system during a specific period. The sliding-window approach enables the model to capture temporal dependencies and behavioural transitions that may indicate anomalous events.

The generated windows are then supplied to the Memory Module, which extracts temporal representations of system behaviour. These representations are used to train the Meta-Hypernetwork and Adaptive Main Network jointly. Through iterative optimization, the model learns to distinguish between normal and anomalous patterns while maintaining the ability to generalize across different operating conditions.

The outcome of the training phase is a set of optimized model parameters that can be deployed on edge devices for real-time anomaly detection.

3.2 Runtime Inference Phase

Once training is completed, the learned model is deployed within the edge environment. During runtime, sensor measurements are continuously received from industrial devices and processed locally. The runtime phase is designed to support low-latency inference while minimizing computational overhead.

Incoming sensor observations are first organized into sequential windows. These windows are forwarded to the Memory Module, which generates contextual representations describing the recent behavioural history of the monitored system. The resulting memory representation is then used by the Meta-Hypernetwork to generate adaptive parameters for the classification network.

The dynamically generated parameters are supplied to the Adaptive Main Network, which produces an anomaly probability score. Based on a predefined decision threshold, the probability score is converted into a binary anomaly label indicating whether the current system behaviour is normal or anomalous.

The use of dynamically generated parameters enables the framework to adapt its decision boundaries according to the current operating environment without requiring complete retraining.

3.3 Memory Module

The Memory Module constitutes the first major component of the proposed architecture. It is implemented using a Long Short-Term Memory (LSTM) network that processes sequential sensor observations and captures temporal dependencies within industrial time-

series data.

Industrial systems exhibit strong temporal relationships where current states often depend on previous operating conditions. Traditional feed-forward models are unable to effectively model such dependencies, making temporal memory learning essential for anomaly detection tasks. The LSTM-based Memory Module addresses this challenge by maintaining hidden states that preserve information across multiple time steps.

The input to the Memory Module consists of normalized sliding windows generated from industrial sensor streams. As each sequence is processed, the LSTM learns temporal patterns representing normal operational behaviour, transitions between states, and recurring behavioural trends.

The output of the Memory Module is a compact memory representation that summarizes the recent history of the monitored system. This representation provides contextual information that is subsequently used by the Meta-Hypernetwork for dynamic parameter generation.

The inclusion of temporal memory significantly improves the model's ability to distinguish between temporary fluctuations and genuine anomalies. It also enhances robustness against sensor noise and transient operational disturbances commonly observed in Industrial IoT environments.

3.4 Meta-Hypernetwork

The Meta-Hypernetwork forms the adaptive component of the proposed framework. Its primary responsibility is to generate dynamic parameters for the Adaptive Main Network based on contextual information received from the Memory Module.

Conventional neural networks operate using fixed parameters learned during training. Although effective in stable environments, such static models often struggle to maintain performance when operating conditions change. The Meta-Hypernetwork addresses this limitation by dynamically generating task-specific parameters during inference.

The memory representation produced by the LSTM serves as input to the Meta-Hypernetwork. Using this contextual information, the hypernetwork generates a set of parameters tailored to the current operating state of the system. These generated parameters are subsequently supplied to the Adaptive Main Network.

Dynamic parameter generation allows the model to modify its behaviour according to

changing environmental conditions, concept drift, and evolving operational patterns. This capability significantly improves adaptability while avoiding the computational cost associated with repeated retraining.

The Meta-Hypernetwork therefore acts as a bridge between temporal memory learning and adaptive anomaly classification, enabling the framework to maintain high detection performance across varying conditions.

3.5 Adaptive Main Network

The Adaptive Main Network performs the final anomaly classification task. Unlike conventional classifiers that rely on fixed weights, the Adaptive Main Network operates using parameters generated dynamically by the Meta-Hypernetwork.

The network receives both the current input window and the generated parameter vector. Using this information, it computes an anomaly probability score that reflects the likelihood of abnormal behaviour within the observed sensor sequence.

The adaptive nature of the classifier enables it to adjust its decision boundaries according to the contextual information captured by the Memory Module. Consequently, the network can respond more effectively to previously unseen patterns and changing operating conditions.

The output produced by the Adaptive Main Network is a probability value ranging between zero and one. Higher values indicate a greater likelihood of anomalous behaviour. A thresholding mechanism is then applied to convert the probability into a binary prediction label.

3.6 Anomaly Prediction Mechanism

The anomaly prediction process combines temporal memory learning, dynamic parameter generation, and adaptive classification. Sensor observations are first converted into sliding windows and passed through the Memory Module. The resulting memory representation is processed by the Meta-Hypernetwork, which generates context-dependent classifier parameters.

The Adaptive Main Network then evaluates the current input sequence using the generated parameters and produces an anomaly probability score. If the probability exceeds a predefined threshold, the sample is classified as anomalous; otherwise, it is classified as normal.

This end-to-end workflow enables real-time anomaly detection while maintaining adaptability and computational efficiency. The architecture eliminates the need for extensive retraining and supports deployment in environments characterized by dynamic behavioural patterns and resource constraints.

3.7 Advantages of the Proposed Framework

The proposed MAMHN architecture offers several advantages over traditional anomaly detection approaches:

1. Temporal memory learning enables effective modelling of sequential industrial sensor data.
2. Dynamic parameter generation improves adaptability under changing operating conditions.
3. The architecture supports few-shot adaptation using limited amounts of new training data.
4. Computational complexity is reduced compared to large transformer-based models.
5. The framework is suitable for deployment in resource-constrained edge computing environments.
6. Real-time anomaly detection can be achieved without continuous cloud connectivity.
7. The model demonstrates strong anomaly detection performance on industrial control system datasets.

The combination of these characteristics makes the proposed Memory-Augmented Meta-Hypernetwork a practical and scalable solution for anomaly detection in modern Industrial IoT systems.

Chapter 4

System Architecture and High-Level Design

The proposed Memory-Augmented Meta-Hypernetwork (MAMHN) architecture is designed to perform adaptive anomaly detection in Industrial Internet of Things (IIoT) environments while maintaining computational efficiency suitable for edge deployment. The architecture combines memory-based temporal learning, dynamic parameter generation, and adaptive classification into a unified framework capable of responding to changing operating conditions and evolving behavioural patterns. The system is organized into two major operational environments:

1.Meta-Training Stage (Cloud / Offline Environment)

2.Edge Device Runtime Environment

The cloud environment is responsible for model training, parameter optimization, and knowledge extraction from historical industrial datasets. The edge environment performs real-time anomaly detection using the trained model while maintaining the ability to adapt to changing operational conditions. Figure 4.1 illustrates the high-level design of the proposed MAMHN framework.

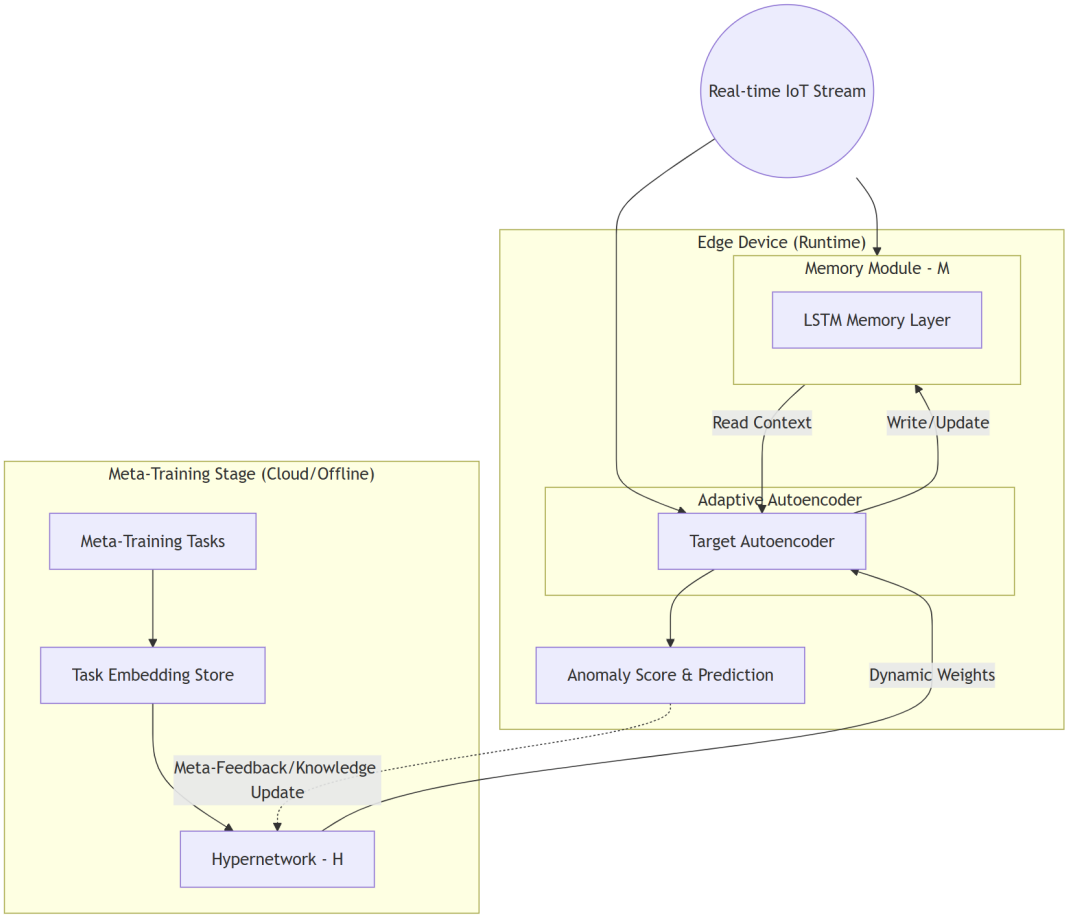

Figure 4.1: High-Level Design of the Proposed MAMHN Framework

4.1 Overall System Workflow

The complete workflow begins with the collection of industrial sensor measurements generated by cyber-physical systems. Historical data collected during normal and anomalous operating conditions is used during the offline training stage to develop generalized behavioural representations.

During runtime, incoming sensor streams are processed locally at the edge node. The Memory Module analyses temporal patterns within the sensor observations and generates contextual information describing the current operating state. This context is forwarded to the Meta-Hypernetwork, which dynamically generates parameters for the Adaptive Main Network.

The Adaptive Main Network then performs anomaly classification and produces anomaly probability scores. These scores are converted into prediction labels that indicate whether

the observed behaviour corresponds to normal system operation or an anomaly.

The architecture is designed to support adaptation through periodic updates and transfer learning mechanisms, enabling continued effectiveness under changing operating conditions.

4.2 Meta-Training Stage

The Meta-Training Stage represents the offline learning environment responsible for training the major components of the MAMHN framework. This stage utilizes historical industrial datasets such as SWaT and WADI to learn generalized behavioural patterns.

The primary objective of the Meta-Training Stage is to create a model capable of transferring knowledge across different operational environments while maintaining high anomaly detection accuracy.

4.2.1 Meta-Training Tasks

Meta-training tasks are generated using industrial sensor data collected under diverse operating conditions. Each task represents a specific behavioural pattern or operating scenario observed within the industrial control system.

The purpose of generating multiple training tasks is to expose the model to a wide variety of behavioural distributions. This enables the framework to learn generalized representations that can be adapted to previously unseen environments.

Through repeated exposure to diverse tasks, the model develops the ability to identify common characteristics associated with normal and anomalous behaviour.

4.2.2 Task Embedding Store

The Task Embedding Store contains compact representations of behavioural patterns extracted during training.

These embeddings summarize important characteristics of industrial operating conditions and provide a structured representation of learned knowledge. The embeddings serve as a foundation for learning generalized behavioural representations and improving transferability across environments.

The stored embeddings are used during optimization of the Meta-Hypernetwork and contribute to the development of adaptive parameter generation strategies.

4.2.3 Meta-Hypernetwork Training

The Meta-Hypernetwork is trained using behavioural representations generated from historical industrial datasets.

The objective of the Meta-Hypernetwork is to learn a mapping between contextual behavioural information and optimal classifier parameters. During training, the hypernetwork learns how different operating conditions influence anomaly classification requirements.

As training progresses, the Meta-Hypernetwork becomes capable of generating parameter sets tailored to specific behavioural contexts. These generated parameters are later utilized by the Adaptive Main Network during runtime inference.

The result of this training process is a dynamic parameter generation mechanism that enables efficient adaptation without requiring complete model retraining.

4.3 Edge Device Runtime Environment

The Edge Device Runtime Environment represents the deployment stage of the MAMHN framework. Once training is completed, the optimized model components are transferred to the edge device where real-time anomaly detection is performed.

The runtime environment is designed to operate under limited computational resources while maintaining high anomaly detection performance.

Sensor observations are continuously processed, analysed, and classified locally, reducing dependence on cloud infrastructure and minimizing communication latency.

4.4 Real-Time Sensor Data Processing

Industrial systems continuously generate multivariate sensor measurements describing the state of physical processes.

These measurements may include:

- Flow rates
- Pressure readings
- Water levels
- Valve positions

- Actuator states
- Network communication indicators

Incoming sensor observations are collected in real time and organized into fixed-length sequential windows before being processed by the learning framework.
The sliding-window representation enables the model to analyse behavioural evolution over time rather than considering individual measurements independently.

4.5 Memory Module

The Memory Module is the first major component operating within the runtime environment.
The module is implemented using an LSTM network that processes sequential sensor observations and captures temporal dependencies present in industrial time-series data.
As input windows are processed, the LSTM generates hidden representations describing recent behavioural history. These representations contain information regarding:

- Normal operating patterns
- Temporal transitions
- Sensor interactions
- Behavioural trends

The generated memory representation serves as contextual information that guides subsequent anomaly classification.
By preserving temporal context, the Memory Module improves robustness against transient fluctuations and sensor noise while enhancing the detection of complex anomalies.

4.6 Context Retrieval and Update Mechanism

The Memory Module continuously updates its internal state as new observations become available.
Each newly observed sensor sequence contributes additional contextual information regarding the current operating condition of the monitored system.
This continuous update process allows the memory representation to evolve naturally as system behaviour changes over time.

Consequently, the model remains sensitive to long-term behavioural trends while retaining awareness of recent operating conditions.

4.7 Meta-Hypernetwork

The Meta-Hypernetwork forms the adaptive core of the MAMHN architecture.

The contextual representation generated by the Memory Module is supplied as input to the Meta-Hypernetwork. Using this information, the network generates a dynamic parameter vector tailored to the current operating state.

Unlike traditional neural networks that utilize fixed parameters during inference, the Meta-Hypernetwork generates parameters dynamically for each behavioural context.

This enables:

- Improved adaptability
- Reduced retraining requirements
- Better handling of concept drift
- Enhanced generalization

The dynamically generated parameters are subsequently transferred to the Adaptive Main Network.

4.8 Adaptive Main Network

The Adaptive Main Network performs anomaly classification using parameters generated by the Meta-Hypernetwork.

The classifier receives both the current sensor representation and the generated parameter vector. These inputs are combined to produce an anomaly probability score.

The adaptive nature of the classifier enables it to modify its decision boundaries according to contextual information extracted from the Memory Module.

This behaviour is particularly beneficial in Industrial IoT environments where operating conditions may change due to maintenance activities, environmental factors, equipment aging, or cyber-attacks.

4.9 Anomaly Prediction Module

The anomaly prediction module converts classifier outputs into final decision labels.

The Adaptive Main Network produces an anomaly probability value ranging from zero to one. This probability indicates the likelihood that the current sensor sequence represents abnormal behaviour.

A thresholding mechanism is applied to convert probabilities into binary labels:

- Normal Behaviour (0)
- Anomalous Behaviour (1)

The generated labels can then be utilized by monitoring systems, dashboards, or automated response mechanisms.

4.10 Knowledge Update and Adaptation

One of the major strengths of the proposed architecture is its ability to support adaptation. As new behavioural patterns emerge, transfer learning and few-shot adaptation techniques can be employed to update the model using limited amounts of additional data.This capability enables the framework to remain effective even when operating conditions evolve significantly beyond those observed during initial training.Periodic retraining can further improve model performance by incorporating newly collected behavioural information into the learning process.

4.11 Advantages of the High-Level Architecture

The proposed high-level architecture provides several important benefits:

1. Real-time anomaly detection capability.
2. Reduced dependence on cloud infrastructure.
3. Dynamic adaptation through hypernetwork-based parameter generation.
4. Efficient modelling of temporal dependencies using LSTM memory.
5. Support for few-shot transfer learning and rapid adaptation.
6. Scalability across diverse industrial environments.
7. Suitability for deployment on resource-constrained edge devices.

These characteristics make the proposed MAMHN architecture an effective solution for adaptive anomaly detection in modern Industrial IoT systems.

Chapter 5

Low-Level Design & Implementation

This chapter presents the detailed low-level design and implementation of the proposed Memory-Augmented Meta-Hypernetwork (MAMHN) framework. While the previous chapter described the high-level architecture and interactions between major subsystems, this chapter focuses on the internal workflow, algorithms, and implementation details used for adaptive anomaly detection. The implementation was developed using Python and PyTorch and consists of several interconnected modules responsible for data preprocessing, temporal memory generation, dynamic parameter generation, adaptive classification, anomaly scoring, and result visualization. Figure 5.1 illustrates the detailed execution workflow of the proposed Memory-Augmented Meta-Hypernetwork (MAMHN) framework. The workflow begins as follows:

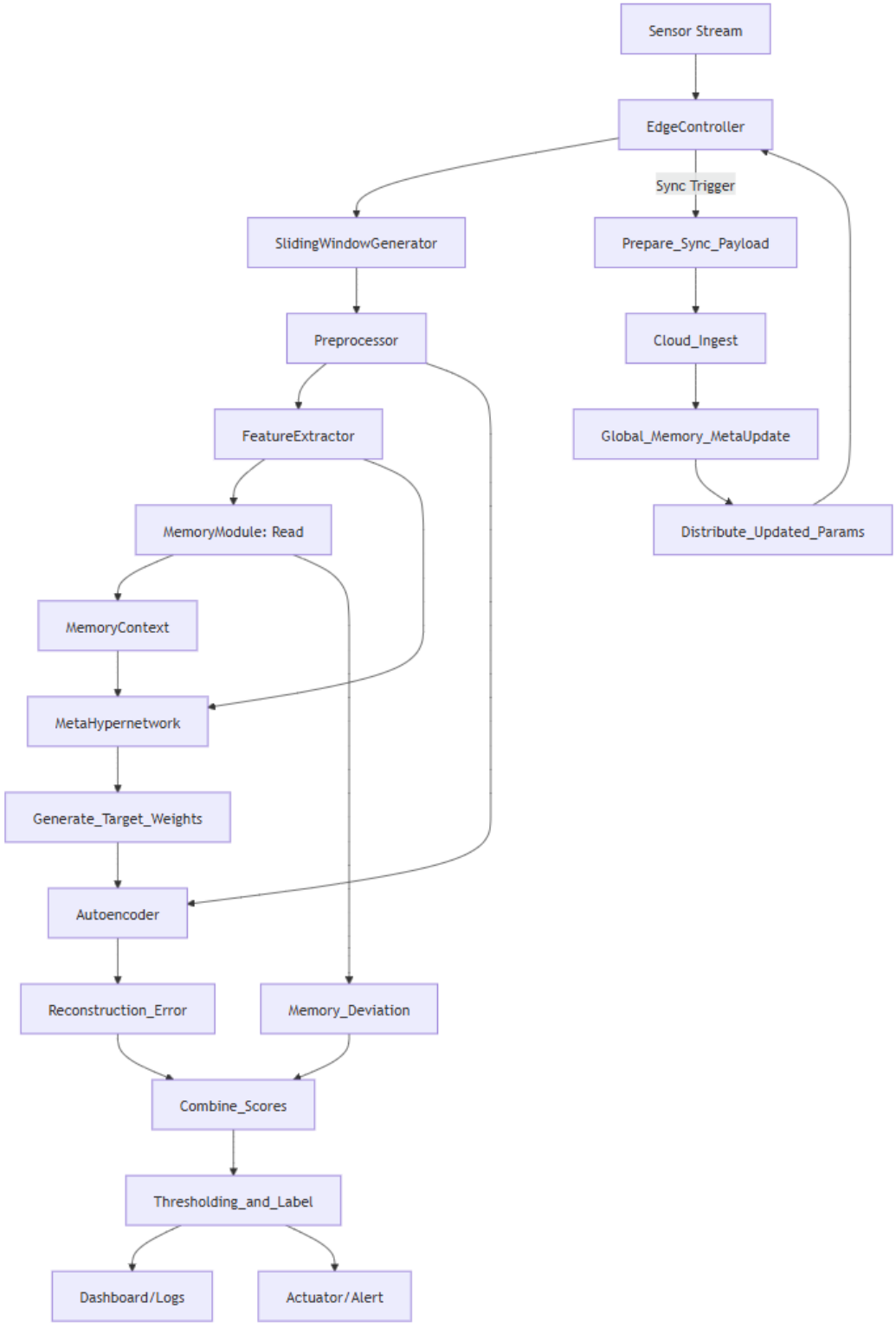

Figure 5.1: Low-Level Design and Execution Workflow of the MAMHN Framework

5.1 Overview of the Execution Pipeline

The MAMHN framework follows a sequential execution pipeline beginning with raw sensor acquisition and ending with anomaly prediction and visualization.

The major stages of execution are:

1. Data Acquisition
2. Data Preprocessing
3. Sliding Window Generation
4. Memory Context Generation
5. Dynamic Parameter Generation
6. Adaptive Classification
7. Anomaly Probability Computation
8. Threshold-Based Decision Making
9. Result Visualization and Export

Each stage contributes to the overall anomaly detection process and is designed to support adaptive learning in dynamic industrial environments.

5.2 Data Acquisition

The first stage of the system involves collecting industrial sensor measurements from historical datasets.

The implementation utilizes industrial control system datasets containing multivariate time-series observations collected from sensors and actuators. Each observation contains multiple process variables describing the current operating condition of the industrial system.

Examples of monitored variables include:

- Water flow measurements
- Tank levels

- Pressure values
- Sensor readings
- Valve states
- Pump conditions

These measurements serve as the raw input to the anomaly detection framework.

5.3 Data Preprocessing

Raw industrial sensor data cannot be directly supplied to neural network models because different variables often exist on different numerical scales.

Therefore, preprocessing is performed before training and inference.

The preprocessing stage consists of:

1. Missing value handling
2. Feature selection
3. Label conversion
4. Normalization

A Min-Max normalization strategy is employed to transform all sensor variables into a common numerical range.

The normalized representation improves training stability and ensures that individual sensor variables do not dominate the learning process due to differences in scale.

5.4 Sliding Window Generation

Industrial sensor data exhibits strong temporal dependencies.

To capture temporal behaviour, the system converts continuous sensor streams into fixed-length sequential windows.

A sliding window mechanism is used to generate overlapping temporal sequences.

For a window length of W:

$$X_i = [x_i, x_{i+1}, x_{i+2}, \ldots, x_{i+W-1}]$$

where:

- X_i represents the generated sequence.
- W represents the window size.

The use of sliding windows allows the model to observe behavioural evolution over time rather than treating individual measurements independently.
This significantly improves anomaly detection performance in industrial environments.

5.5 Memory Module Implementation

The Memory Module is implemented using a Long Short-Term Memory (LSTM) network.
The primary purpose of this component is to capture temporal relationships present within sequential sensor observations.
The LSTM receives the generated sliding windows as input and processes each timestep sequentially.
During execution, the LSTM maintains hidden states and cell states that preserve information from previous observations.
The internal operations of the LSTM can be summarized using the following equations:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$h_t = o_t \odot \tanh(C_t)$$

The final hidden representation generated by the LSTM serves as the memory context vector.
This vector summarizes recent behavioural patterns observed within the industrial process.

5.6 Context Representation Generation

The hidden representation produced by the Memory Module acts as a compact behavioural embedding.

This embedding captures:

- Recent operational history
- Sensor interactions
- Behavioural transitions
- Temporal dependencies

The generated context vector provides a concise description of the current system state and serves as input to the Meta-Hypernetwork.

5.7 Meta-Hypernetwork Implementation

The Meta-Hypernetwork forms the adaptive core of the proposed framework.

Instead of directly performing classification, the hypernetwork generates parameters for another neural network.

The memory representation produced by the LSTM is supplied as input.

The Meta-Hypernetwork processes this representation through fully connected layers and generates a dynamic parameter vector.

The generated parameters include:

- Dynamic weight matrices
- Dynamic bias vectors

These parameters are specifically tailored to the current behavioural context.

Consequently, the final classifier can modify its behaviour according to the observed operating conditions.

5.8 Adaptive Main Network

The Adaptive Main Network receives two inputs:

1. Current sensor features

2. Dynamic parameters generated by the Meta-Hypernetwork

The classifier applies the generated parameters to the incoming feature representation and computes anomaly scores.

Unlike conventional classifiers that rely on fixed parameters, the Adaptive Main Network continuously adapts its decision boundaries according to the context provided by the Memory Module.

This dynamic behaviour enables improved robustness under changing operational conditions.

5.9 Anomaly Probability Computation

The output of the Adaptive Main Network is transformed into an anomaly probability using the Sigmoid activation function.

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

where:

- z represents the classifier output.
- $P(y = 1)$ represents anomaly probability.

The probability value ranges between 0 and 1 and indicates the likelihood of abnormal behaviour.

Higher values correspond to a greater probability of anomalies.

5.10 Threshold-Based Decision Module

The anomaly probability is converted into a binary prediction using threshold-based decision making.

The decision rule is defined as:

$$\text{Prediction} = \begin{cases} 1, & P(y = 1) > Threshold \\ 0, & Otherwise \end{cases}$$

where:

- 1 indicates an anomaly.

- 0 indicates normal behaviour.

This module enables flexible sensitivity adjustment according to deployment requirements.

5.11 Few-Shot Adaptation Mechanism

A key feature of the proposed framework is its ability to perform rapid adaptation using limited data.

During adaptation:

1. Memory Module parameters remain fixed.
2. Meta-Hypernetwork parameters remain fixed.
3. Only the Adaptive Main Network is updated.

This strategy significantly reduces training time and computational overhead.

The model can therefore adapt to new industrial environments using only a small fraction of available training samples.

5.12 Dashboard and Visualization Module

To improve usability, a Streamlit-based dashboard is implemented.

The dashboard provides:

- CSV upload functionality
- Threshold selection
- Window-size configuration
- Anomaly probability visualization
- Prediction distribution plots
- Result export functionality

These visualizations allow operators to monitor system behaviour and investigate anomalous events.

5.13 Output Generation

The final stage of execution generates:

- Anomaly probabilities
- Binary prediction labels
- Evaluation metrics
- Downloadable result files

Performance metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix values are calculated to assess model effectiveness.
The generated outputs provide both quantitative performance evaluation and operational decision support for industrial monitoring systems.

5.14 Summary

The low-level implementation of the proposed MAMHN framework combines temporal memory learning, dynamic parameter generation, adaptive classification, and few-shot adaptation into a unified anomaly detection pipeline. The integration of these components enables efficient real-time anomaly detection while maintaining adaptability to changing industrial operating conditions. The modular design further supports scalability, maintainability, and deployment in resource-constrained edge environments.

Chapter 6

Software Architecture Design

This chapter presents the software architecture of the proposed Memory-Augmented Meta-Hypernetwork (MAMHN) framework. The software architecture defines the structural organization of the system and illustrates the interaction between the major modules involved in adaptive anomaly detection. A class diagram is used to represent the relationships among different software components and to provide a static view of the overall system design. The class diagram shown in Figure 6.1 illustrates the major software entities of the proposed MAMHN framework and their interactions. The architecture follows a modular design approach in which each component is responsible for a specific stage of the anomaly detection pipeline. This modular organization improves maintainability, scalability, and ease of future extension.

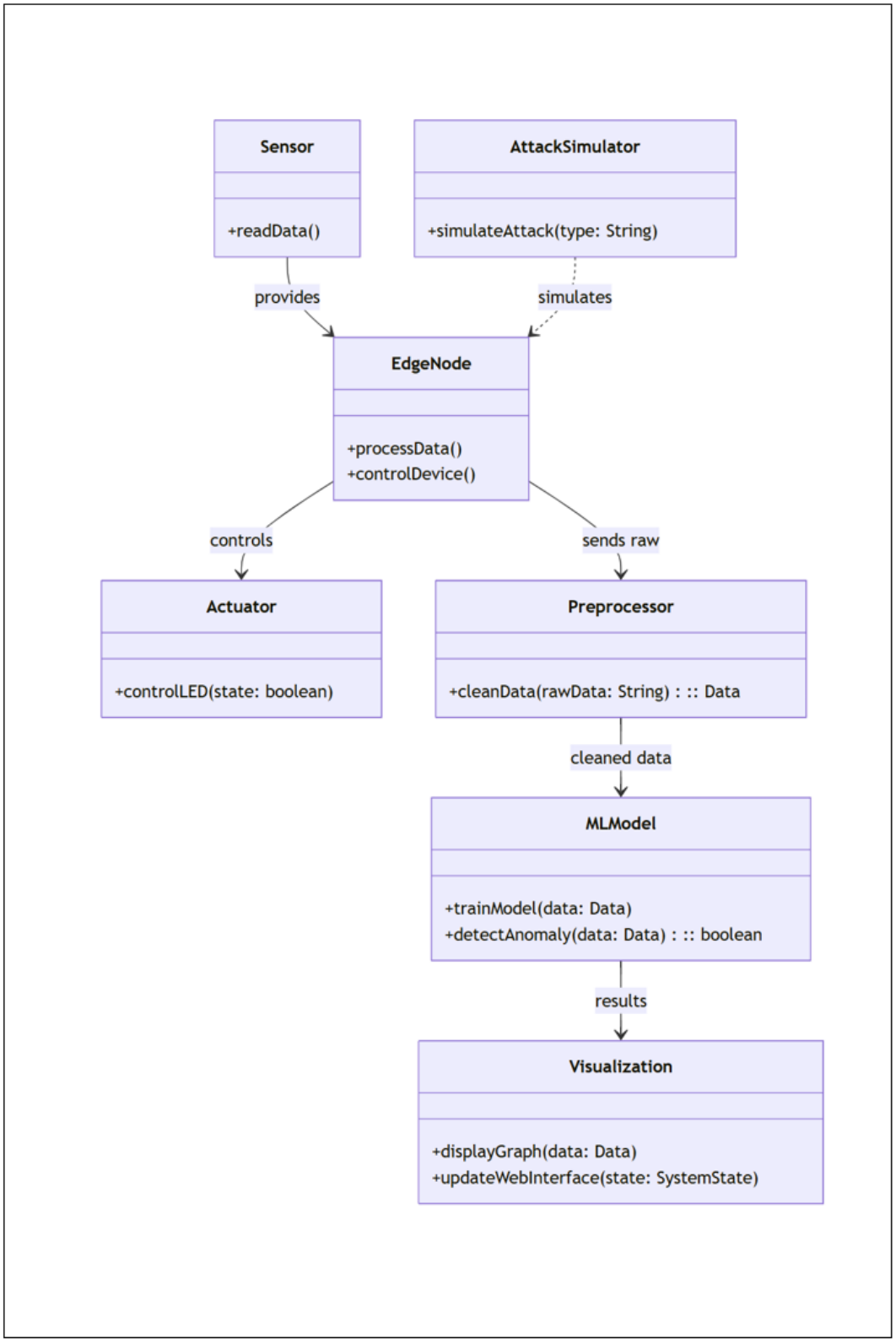

Figure 6.1: Class Diagram of the Proposed MAMHN Framework

6.1 Class Diagram Explanation

The class diagram represents the software components required for processing industrial time-series data, generating adaptive model parameters, and performing anomaly detection. Each class contributes to a specific stage of the overall workflow.

6.1.1 SensorStream Class

The SensorStream class acts as the entry point of the framework. It is responsible for acquiring raw sensor observations from industrial datasets and forwarding them to subsequent processing modules. The class abstracts the data source and provides a uniform interface for accessing sensor measurements.

6.1.2 SlidingWindowGenerator Class

The SlidingWindowGenerator converts continuous sensor streams into fixed-length temporal sequences. These windows preserve temporal dependencies and allow the model to analyse behavioural evolution over time. The generated windows serve as the primary input for the learning modules.

6.1.3 Preprocessor Class

The Preprocessor class performs data cleaning, normalization, and feature preparation. Sensor variables often exist on different numerical scales; therefore, normalization is required to improve training stability and model performance. The preprocessed output is forwarded to the Memory Module.

6.1.4 MemoryModule Class

The MemoryModule is implemented using a Long Short-Term Memory (LSTM) network. It captures temporal dependencies within sequential sensor observations and generates contextual memory representations describing recent system behaviour. These representations provide temporal context for adaptive parameter generation.

6.1.5 MetaHypernetwork Class

The MetaHypernetwork receives memory representations generated by the MemoryModule and produces dynamic parameter vectors. Unlike conventional neural networks that use fixed weights during inference, the MetaHypernetwork generates context-aware pa-

rameters that adapt according to the current operating environment.

6.1.6 AdaptiveMainNetwork Class

The AdaptiveMainNetwork performs the final anomaly classification task. It utilizes the dynamically generated parameters supplied by the MetaHypernetwork to classify incoming sensor windows as either normal or anomalous. This adaptive mechanism enables improved performance under changing operating conditions.

6.1.7 AnomalyDetector Class

The AnomalyDetector class receives the output probabilities generated by the AdaptiveMainNetwork and applies threshold-based decision making. Based on the configured threshold value, the class generates the final anomaly prediction label.

6.1.8 Dashboard Class

The Dashboard class provides an interactive user interface for monitoring model outputs. It enables users to upload datasets, configure detection parameters, visualize anomaly predictions, and download generated results. The dashboard improves usability and facilitates practical deployment of the framework.

6.2 Relationship Among Classes

The SensorStream class provides input data to the SlidingWindowGenerator. The generated windows are processed by the Preprocessor before being forwarded to the MemoryModule. The contextual representations generated by the MemoryModule are supplied to the MetaHypernetwork, which produces dynamic classifier parameters. These parameters are utilized by the AdaptiveMainNetwork to perform anomaly classification. Finally, the AnomalyDetector generates prediction labels, which are displayed through the Dashboard interface.

The interaction among these classes establishes a complete anomaly detection pipeline capable of adaptive learning and real-time prediction. The modular structure also allows individual components to be upgraded or replaced without affecting the overall architecture.

6.3 Summary

The software architecture of the proposed MAMHN framework follows a modular design that separates data acquisition, preprocessing, memory learning, parameter generation, classification, and visualization into distinct software components. The class diagram provides a clear representation of these relationships and demonstrates how the different modules cooperate to achieve adaptive anomaly detection in Industrial Internet of Things environments.

Chapter 7

System Implementation

This chapter presents the implementation details of the proposed Memory-Augmented Meta-Hypernetwork (MAMHN) framework. The implementation was carried out using Python and PyTorch, with supporting libraries for data processing, visualization, and deployment. The objective of the implementation phase was to transform the proposed architecture into a functional anomaly detection system capable of processing industrial time-series data and generating real-time anomaly predictions.

7.1 Development Environment

The proposed framework was implemented using a software-based environment that supports machine learning, deep learning, and interactive visualization.
The development environment consisted of:

- Python 3.x
- PyTorch
- NumPy
- Pandas
- Scikit-Learn
- Matplotlib
- Seaborn
- Streamlit
- Google Colab with NVIDIA T4 GPU

Python was selected because of its extensive support for machine learning and data science applications. PyTorch was used for constructing and training the neural network components of the proposed architecture.

7.2 Hardware Configuration

The experiments were conducted using Google Colab with an NVIDIA T4 GPU accelerator.

Listing 7.1: GPU Configuration

```
device = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "cpu"
)
```

GPU acceleration significantly reduced training time and enabled efficient experimentation on large industrial datasets.

7.3 Dataset Preparation

Industrial control system datasets were used for training and evaluation. The datasets consisted of multivariate time-series sensor measurements collected from industrial processes operating under both normal and anomalous conditions.
The dataset preparation stage involved:

1. Loading raw CSV files.
2. Removing unnecessary attributes.
3. Handling missing values.
4. Converting labels into binary classes.
5. Splitting data into training and testing subsets.

The prepared datasets were subsequently forwarded to the preprocessing pipeline.

7.3.1 Dataset Loading Code

The industrial datasets were loaded using the Pandas library.

Listing 7.2: Dataset Loading

```
import pandas as pd

df = pd.read_csv(
    "dataset.csv"
)

X = df.drop(
    "Label",
    axis=1
)

y = df["Label"]
```

The extracted feature matrix and labels are subsequently used for training and evaluation.

7.4 Data Preprocessing Implementation

Before training, the sensor measurements were normalized using Min-Max scaling.

The normalization process transforms feature values into a common numerical range and improves optimization stability.

The normalized feature value is computed as:

$[X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}]$

where:

- X represents the original feature value.
- X_{min} represents the minimum feature value.
- X_{max} represents the maximum feature value.

The preprocessing stage also ensured that all features were represented in a format compatible with the neural network architecture.

7.4.1 Normalization Code

Listing 7.3: Feature Normalization

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)
```

The Min-Max scaler transforms all sensor features into a common numerical range and improves training stability.

7.5 Sliding Window Generation

A sliding window mechanism was implemented to capture temporal dependencies present in industrial sensor streams.

For a window length W, each generated sequence is represented as:

[$X_i = [x_i, x_{i+1}, x_{i+2}, ..., x_{i+W-1}]$]

The sliding windows provide sequential context and enable the model to analyse behavioural evolution over time.

The generated windows were stored as tensors and supplied to the learning framework.

7.5.1 Sliding Window Generation Code

Listing 7.4: Sliding Window Generation

```
def create_windows(
    data,
    window_size
):

    windows = []

    for i in range(
        len(data)-window_size
    ):
```

```
        windows.append(
            data[i:i+window_size]
        )

    return np.array(windows)
```

The generated windows preserve temporal relationships between sensor observations and form the primary input to the learning framework.

7.6 Memory Module Implementation

7.6.1 Memory Module Code Snippet

The Memory Module is implemented using an LSTM network that captures temporal dependencies within sequential sensor observations.

Listing 7.5: LSTM Memory Module

```
class MemoryModule(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.lstm = nn.LSTM(
            input_dim,
            hidden_dim,
            batch_first=True
        )

    def forward(self, x):
        _, (hidden, _) = self.lstm(x)
        return hidden.squeeze(0)
```

The Memory Module was implemented using a Long Short-Term Memory (LSTM) network.

The module receives sequential windows and generates contextual behavioural representations.

The implementation consists of:

- Input Layer

- LSTM Layer
- Hidden State Generation
- Context Representation Extraction

The hidden state generated by the LSTM serves as the memory representation describing recent system behaviour.
This representation captures temporal relationships and provides contextual information for subsequent adaptive learning.

7.7 Meta-Hypernetwork Implementation

7.7.1 Meta-Hypernetwork Code Snippet

The Meta-Hypernetwork generates dynamic classifier parameters from contextual memory representations.

Listing 7.6: Meta-Hypernetwork

```
class MetaHyperNetwork(nn.Module):
    def __init__(self, hidden_dim, output_dim):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(hidden_dim, 128),
            nn.ReLU(),
            nn.Linear(128, output_dim)
        )

    def forward(self, context):
        return self.network(context)
```

The Meta-Hypernetwork was implemented using fully connected neural network layers. Its primary function is to generate dynamic parameter vectors based on the contextual memory representation produced by the Memory Module.
The implementation process consists of:

1. Receiving contextual memory representation.

2. Processing representation through dense layers.

3. Generating dynamic parameter vectors.

4. Transferring generated parameters to the Adaptive Main Network.

The generated parameters enable the classifier to adapt to changing operating conditions without requiring complete retraining.

7.8 Adaptive Main Network Implementation

7.8.1 Adaptive Main Network Code Snippet

The Adaptive Main Network performs anomaly classification using dynamically generated parameters.

Listing 7.7: Adaptive Main Network

```
class AdaptiveMainNetwork(nn.Module):
    def __init__(self, input_dim):
        super().__init__()

        self.fc = nn.Linear(input_dim, 1)

    def forward(self, x):
        return torch.sigmoid(self.fc(x))
```

The Adaptive Main Network performs the final anomaly classification task.
The network receives:

- Current sensor representation.

- Dynamic parameters generated by the Meta-Hypernetwork.

The implementation combines these inputs to compute anomaly probabilities.
The Adaptive Main Network continuously adjusts its behaviour according to the generated parameters, improving robustness under concept drift and changing environments.

7.9 Training Procedure

7.9.1 Training Loop Implementation

The model parameters are optimized using the Adam optimizer.

Listing 7.8: Training Loop

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)

for epoch in range(num_epochs):

    optimizer.zero_grad()

    outputs = model(inputs)

    loss = criterion(outputs, labels)

    loss.backward()

    optimizer.step()
```

The complete MAMHN architecture was trained using supervised learning.
The training process consisted of:

1. Forward propagation.
2. Loss computation.
3. Backpropagation.
4. Parameter optimization.

The Adam optimizer was employed to update network parameters due to its fast convergence and stability.
Training was performed over multiple epochs until performance convergence was achieved.

7.10 Few-Shot Adaptation Mechanism

One of the key objectives of the proposed framework is rapid adaptation using limited data.

To achieve this, a few-shot adaptation mechanism was implemented.

During adaptation:

- The Memory Module remains fixed.
- The Meta-Hypernetwork remains fixed.
- Only the final classification layers are updated.

This strategy significantly reduces computational overhead and enables rapid adaptation to new environments.

7.11 Model Deployment Workflow

The deployment workflow consists of:

1. Uploading sensor data.
2. Preprocessing and normalization.
3. Sliding window generation.
4. Memory representation extraction.
5. Dynamic parameter generation.
6. Anomaly classification.
7. Visualization of results.

This workflow enables practical deployment of the proposed MAMHN framework for industrial anomaly detection applications.

7.12 Summary

The implementation of the proposed MAMHN framework successfully integrates temporal memory learning, dynamic parameter generation, adaptive classification, and few-shot adaptation into a unified anomaly detection system. The use of PyTorch enabled efficient development and training of the model, while the modular architecture supports scalability, adaptability, and future enhancement for Industrial Internet of Things environments.

Chapter 8

Results and Analysis

This chapter presents the experimental evaluation of the proposed Memory-Augmented Meta-Hypernetwork (MAMHN) framework. The objective of the evaluation is to analyse the anomaly detection performance, adaptability, and deployment suitability of the proposed model using industrial time-series datasets. Standard classification metrics and confusion matrix analysis are employed to assess the effectiveness of the framework.

8.1 Experimental Setup

The experiments were conducted using Google Colab with NVIDIA T4 GPU acceleration. The proposed framework was implemented using Python and PyTorch. The evaluation was performed on industrial control system datasets containing both normal and anomalous operational behaviour.

The experimental objectives were:

- Evaluate anomaly detection performance.
- Measure classification accuracy and robustness.
- Analyse false positive and false negative behaviour.
- Compare the proposed framework with modern anomaly detection approaches.
- Assess suitability for Industrial IoT edge deployment.

8.2 Evaluation Metrics

To evaluate the effectiveness of the proposed framework, standard classification metrics were used.

8.2.1 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

8.2.2 Precision

$$mathrmPrecision = \frac{TP}{TP + FP}$$

8.2.3 Recall

$$mathrmRecall = \frac{TP}{TP + FN}$$

8.2.4 F1-Score

$$mathrmF1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

8.3 Performance Results

The proposed MAMHN framework achieved strong anomaly detection performance on the industrial datasets.

Table 8.1: Performance Metrics of MAMHN

Metric	Value
Accuracy	90.59%
Precision	90.21%
Recall	98.29%
F1-Score	94.08%

The obtained results indicate that the proposed framework is highly effective in identifying anomalous behaviour. The high recall value demonstrates that the model successfully detects the majority of anomaly instances while minimizing missed detections.

The F1-score of 94.08

8.4 Confusion Matrix Analysis

The confusion matrix obtained during evaluation is shown in Table 8.2.

Table 8.2: Confusion Matrix

	Predicted Normal	Predicted Anomaly
Actual Normal	2270	1161
Actual Anomaly	186	10697

The model correctly identified 10,697 anomaly samples while missing only 186 anomaly instances. This demonstrates strong anomaly sensitivity and confirms the effectiveness of the memory-guided adaptation mechanism.

Although some false positives were generated, prioritizing anomaly detection is often desirable in industrial environments where missing a critical anomaly may result in operational or safety risks.

8.5 Comparison with Existing Methods

The proposed MAMHN framework was compared conceptually with modern anomaly detection approaches including TranAD and TimesNet.

Table 8.3: Comparison of Anomaly Detection Frameworks

Method	Adaptability	Computational Cost	Edge Suitability
TranAD	High	High	Moderate
TimesNet	High	High	Moderate
MAMHN	High	Low	High

TranAD employs transformer-based self-attention mechanisms to model temporal relationships. Although highly accurate, transformer architectures require significant computational resources and memory.

TimesNet utilizes frequency-domain analysis and two-dimensional convolutional operations to extract temporal patterns. While effective, its computational requirements increase deployment complexity on low-power edge devices.

In contrast, the proposed MAMHN framework achieves adaptability through memory-guided parameter generation and temporal representation learning while maintaining

lower computational overhead, making it more suitable for Industrial IoT edge deployment.

8.6 Edge Deployment Analysis

One of the major objectives of the proposed framework is deployment feasibility on resource-constrained edge devices.

The use of an LSTM-based Memory Module and lightweight Meta-Hypernetwork significantly reduces computational complexity compared with transformer-based approaches. Dynamic parameter generation enables adaptation without complete retraining, reducing both memory consumption and operational cost.

These characteristics make the framework suitable for practical deployment in Industrial Internet of Things environments where computational resources are limited.

8.7 Discussion

The experimental results demonstrate that integrating temporal memory learning with dynamic parameter generation improves anomaly detection performance while maintaining computational efficiency.

The high recall value indicates that the proposed framework successfully identifies anomalous behaviour across diverse operating conditions. The few-shot adaptation capability further improves robustness under changing environments and concept drift.

Overall, the results validate the effectiveness of combining memory augmentation, meta-learning principles, and hypernetwork-based parameter generation within a unified anomaly detection framework.

8.8 Summary

The proposed MAMHN framework achieved an accuracy of 90.59%, precision of 90.21%, recall of 98.29%, and F1-score of 94.08%. The confusion matrix analysis confirms strong anomaly detection capability, while the comparison with existing methods highlights the suitability of the framework for adaptive anomaly detection on Industrial IoT edge devices.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

This project presented a Memory-Augmented Meta-Hypernetwork (MAMHN) framework for adaptive anomaly detection in Industrial Internet of Things (IIoT) environments. The proposed architecture was designed to address the limitations of conventional anomaly detection systems, particularly their inability to adapt efficiently to changing operational conditions and evolving data distributions.

The framework integrates three key components: a Memory Module based on Long Short-Term Memory (LSTM) networks, a Meta-Hypernetwork for dynamic parameter generation, and an Adaptive Main Network for anomaly classification. By combining temporal memory learning with adaptive parameter generation, the proposed model is capable of capturing sequential behavioural patterns while adjusting its decision boundaries according to contextual information.

The implementation was carried out using Python and PyTorch, and the model was evaluated using industrial time-series datasets. Experimental results demonstrated the effectiveness of the proposed approach, achieving an accuracy of 90.59

Furthermore, the use of lightweight neural components makes the framework suitable for deployment in resource-constrained edge computing environments. The integration of few-shot adaptation mechanisms also enables rapid learning from limited data, improving flexibility and reducing retraining requirements.

Overall, the results validate the effectiveness of the proposed Memory-Augmented Meta-Hypernetwork architecture as a practical solution for adaptive anomaly detection in Industrial IoT systems.

9.2 Future Work

Although the proposed framework demonstrates strong anomaly detection performance, several opportunities exist for further enhancement and research.

1. **Federated Learning Integration**

   ‘‘‘ Future versions of the framework can incorporate federated learning to enable collaborative model improvement across multiple edge devices while preserving data privacy.

2. **Online Continual Learning**

   The current implementation performs adaptation using predefined training procedures. Future research may investigate fully online continual learning mechanisms capable of updating the model continuously during deployment.

3. **Advanced Memory Management**

   More sophisticated memory update and retrieval strategies can be explored to improve long-term behavioural retention and reduce memory redundancy.

4. **Lightweight Edge Optimization**

   Model compression techniques such as pruning, quantization, and knowledge distillation can be applied to further reduce computational overhead and enable deployment on extremely resource-constrained devices.

5. **Multi-Domain Evaluation**

   Future studies may evaluate the framework on a broader range of datasets including smart manufacturing, healthcare monitoring, transportation systems, and cybersecurity applications.

6. **Hybrid Attention Mechanisms**

   Integrating lightweight attention mechanisms with the existing memory architecture may improve the ability of the framework to capture complex long-range dependencies while maintaining efficiency.

7. **Real-Time Industrial Deployment**

   The ultimate extension of this work involves deployment and validation within real industrial environments to assess performance under practical operating conditions and large-scale streaming data scenarios. ‘‘‘

9.3 Final Remarks

The increasing adoption of Industrial IoT technologies demands anomaly detection systems that are accurate, adaptive, and computationally efficient. The proposed Memory-Augmented Meta-Hypernetwork framework demonstrates that combining temporal memory learning, meta-learning principles, and dynamic parameter generation can effectively address these requirements. The results obtained in this work provide a strong foundation for future research into adaptive edge intelligence and real-time industrial anomaly detection systems.

Bibliography

[1] S. Jena, A. Pulkit, K. Singh, A. Banerjee, S. Joshi, A. Ganesh, D. Singh, and A. Bhavsar, "Unified Anomaly Detection Methods on Edge Devices Using Knowledge Distillation and Quantization," *arXiv e-Print*, 2024.

[2] Z. Alwaisi, T. Kumar, E. Harjula, and S. Soderi, "A Lightweight Machine Learning Approach for Anomaly Detection and Prevention in Constrained IoT Systems," *Internet of Things*, 2024.

[3] M. Antonini, M. Pincheira, M. Vecchio, and F. Antonelli, "An Adaptable and Unsupervised TinyML Anomaly Detection System for Extreme Industrial Environments," *Sensors*, 2023.

[4] D. K. Singh, D. R. Pant, G. Gautam, and B. Shrestha, "Meta-Learning Approach for Adaptive Anomaly Detection from Multi-Scenario Video Surveillance," *Applied Sciences*, 2025.

[5] J. Liu, X. Zhao, Y. Luo, S. Qin, Q. Fu, and H. Tan, "Memory-Augmented Meta-Learning Framework for Cross-Subject Motor Imagery Classification," *Biomedical Signal Processing and Control*, 2025.

[6] X. Kang, Y. Li, Y. Zhang, N. Ma, and L. Wen, "Anomaly Detection in Concrete Dams Using Memory-Augmented Autoencoder and GAN (MemAE-GAN)," *Automation in Construction*, 2024.

[7] A. Santoro, S. Bartunov, M. Botvinick, D. Wierstra, and T. Lillicrap, "Meta-Learning with Memory-Augmented Neural Networks," *Proceedings of ICML*, 2016.

[8] D. Zhao, S. Kobayashi, J. Sacramento, and J. von Oswald, "Meta-Learning via Hypernetworks," *NeurIPS Meta-Learning Workshop*, 2020.

[9] V. K. Chauhan, J. Zhou, P. Lu, S. Molaei, and D. A. Clifton, "A Brief Review of Hypernetworks in Deep Learning," *Artificial Intelligence Review*, 2024.

[10] Y. Shin, K. Lee, S. Lee, Y. R. Choi, H.-S. Kim, and J. Ko, "Efficient Hypernetwork-Based Weight Generation for Heterogeneous Federated Learning," *Proceedings of ACM SenSys*, 2024.

[11] J. Hao, P. Chen, J. Chen, and X. Li, "Unsupervised Federated Hypernetwork for Detecting Distributed Multivariate Time-Series Anomalies," *Information Processing and Management*, 2025.

[12] C. Zhang, T. Jia, G. Shen, P. Zhu, and Y. Li, "MetaLog: Generalizable Cross-System Anomaly Detection from Logs Using Meta-Learning," *IEEE/ACM ICSE*, 2024.

[13] S. Wang, K. Ding, C. Zhang, C. Chen, and J. Li, "Task-Adaptive Few-Shot Node Classification," *Proceedings of ACM KDD*, 2022.

[14] L. Gudala, M. Shaik, S. Venkataramanan, and A. K. R. Sadhu, "AI-Based Threat Detection and Anomaly Identification in Resource-Constrained IoT Networks," *Distributed Learning and Applications*, 2019.

[15] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data," *Proceedings of the VLDB Endowment*, vol. 15, no. 6, pp. 1201–1214, 2022.

[16] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

Appendices

Appendix A

Sustainable Development Goals Addressed

#	**SDG**	**Level**
1	No Poverty	1
2	Zero Hunger	1
3	Good Health and Well-being	2
4	Quality Education	3
5	Gender Equality	1
6	Clean Water and Sanitation	1
7	Affordable and Clean Energy	2
8	Decent Work and Economic Growth	2
9	Industry, Innovation and Infrastructure	3
10	Reduced Inequalities	1
11	Sustainable Cities and Communities	3
12	Responsible Consumption and Production	2
13	Climate Action	2
14	Life Below Water	1
15	Life on Land	1
16	Peace, Justice and Strong Institutions	1
17	Partnerships for the Goals	2

Levels: Poor = 1, Good = 2, Excellent = 3

Appendix B

Self-Assessment of the Project

Programme Outcomes (POs)

#	**PO**	**Contribution from the Project**	**Level**
1	Engineering Knowledge	Applied ML, meta-learning, LSTM-based memory modules and IoT preprocessing techniques to build an adaptive anomaly detection system.	3
2	Problem Analysis	Analysed IoT datasets to identify behavioural deviations, spikes, and anomaly patterns.	3
3	Design and Development of Solutions	Designed a Memory-Augmented Meta-Hypernetwork (MAMHN) for cross-domain anomaly detection.	3
4	Investigation of Complex Problems	Evaluated system performance using F1-score, AUC, confusion matrix and retraining strategies.	3
5	Modern Tool Usage	Used PyTorch, Jupyter Notebook, Streamlit dashboards and scalable ML pipelines.	3
6	The Engineer and the World	Supported smart infrastructure by detecting IoT faults in real-time sensor streams.	2

#	**PO**	**Contribution from the Project**	**Level**
7	Ethics	Ensured responsible dataset usage, privacy protection and transparent anomaly reporting.	2
8	Individual and Team Work	Worked effectively in a two-member team with shared responsibilities across development and evaluation.	3
9	Communication	Presented outcomes through dashboards, documentation, visual analytics and formal presentations.	3
10	Project Management and Finance	Managed datasets, retraining cycles, computational resources and project timelines.	2
11	Life-long Learning	Explored advanced architectures and emerging concepts in IoT anomaly detection.	3

Levels: Poor = 1, Good = 2, Excellent = 3

Appendix C

Data Sheet of Component 1

Component 1: SWaT Dataset (Secure Water Treatment)

The Secure Water Treatment (SWaT) dataset contains multivariate time-series data collected from a scaled-down operational water treatment plant. The dataset includes both normal operational behaviour and multiple cyber-attack scenarios, making it one of the most widely used benchmarks for industrial anomaly detection.

The key parameters utilized in this project include:

- Water Level Sensors (LIT)
- Flow Rate Sensors (FIT)
- Pressure Sensors (PIT)
- Motorized Valve States (MV)
- Pump Status Indicators (P)
- Water Quality Measurements
- Actuator States
- Timestamp Information
- Binary Attack Labels

Relevance: Provides realistic industrial control system data containing both normal and anomalous operational behaviour.

Reason for Selection: SWaT is a standard benchmark dataset extensively used in Industrial IoT and cyber-physical system anomaly detection research. It enables evaluation of anomaly detection performance under realistic industrial attack scenarios.

Appendix D

Data Sheet of Component 2

Component 2: Pump Sensor Dataset

The Pump Sensor Dataset contains multivariate time-series measurements collected from industrial pump monitoring systems. The dataset consists of sensor readings that capture the operational behaviour of pumps under normal and abnormal conditions. The data is commonly used for predictive maintenance, fault diagnosis, and anomaly detection research.

The key parameters utilized in this project include:

- Sensor_1
- Sensor_2
- Sensor_3
- Sensor_4
- Sensor_5
- Sensor_6
- Sensor_7
- Sensor_8
- Sensor_9
- Sensor_10
- Sensor_11
- Sensor_12
- Sensor_13
- Sensor_14
- Sensor_15

- Sensor_16
- Sensor_17
- Sensor_18
- Sensor_19
- Sensor_20
- Sensor_21
- Timestamp
- Machine Status Label

Relevance: Provides real-world industrial sensor measurements for evaluating anomaly detection performance under equipment degradation and fault conditions.

Reason for Selection: The dataset represents a realistic predictive maintenance scenario and contains complex multivariate sensor relationships that are suitable for evaluating the adaptive learning capabilities of the proposed Memory-Augmented Meta-Hypernetwork (MAMHN) framework.